📘 Day 8: Recurrent Neural Network (RNN)

🔴 First, What's Wrong with ANN?

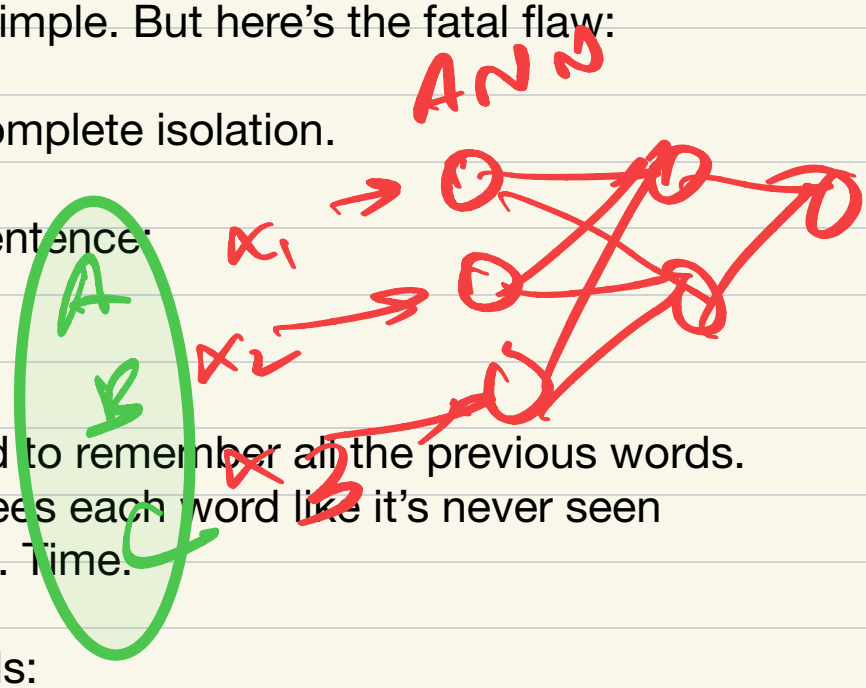
A standard Artificial Neural Network takes an input, passes it through layers, and gives an output. Simple. But here's the fatal flaw:

Every input is processed in complete isolation.

Imagine you're reading this sentence:

"The cat sat on the ___"

To predict the blank, you need to remember all the previous words. But ANN has no memory. It sees each word like it's never seen anything before. Every. Single. Time.



Real problems where ANN fails:

- Predicting the next word in a sentence
- Translating a language
- Recognizing speech
- Stock price forecasting

The root cause → ANN has no concept of time or order.

🟣 ANN Architecture (Why No Memory)

Input Layer Hidden Layer Output Layer

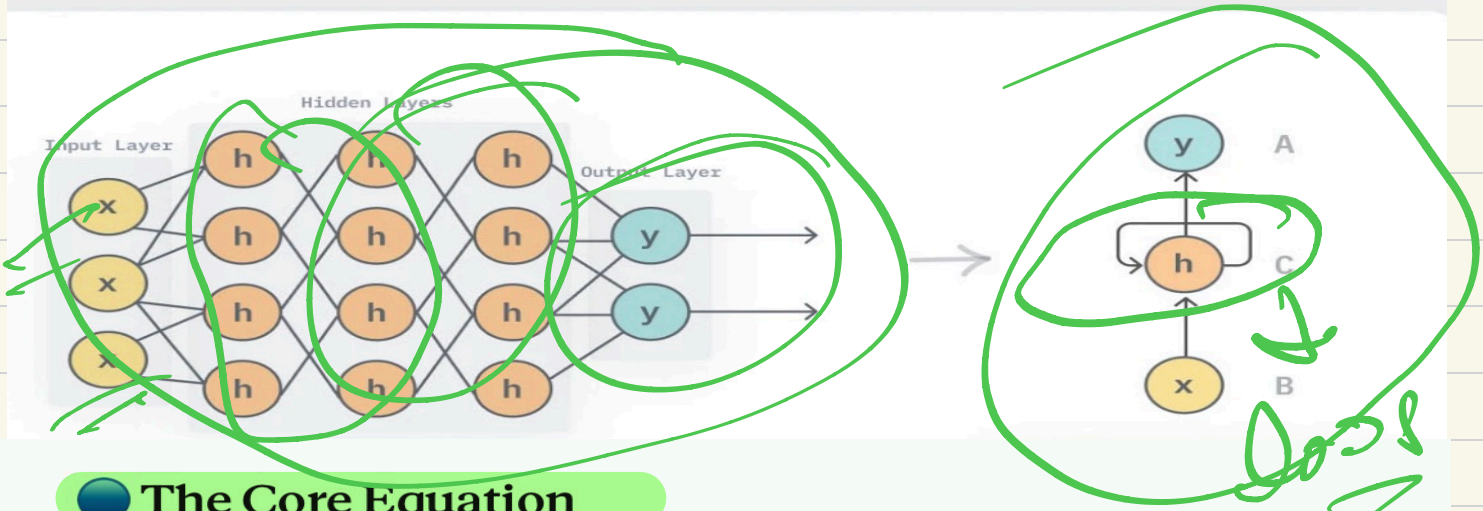
x_1, x_2, x_3 → **[Neurons]** → $\hat{y}$

Each input x is processed FRESH. No connection to past inputs.

Enter RNN — The Memory Machine

RNN adds one revolutionary idea: a hidden state h that gets passed from one step to the next.

Think of it like short-term memory. After reading each word, the network updates its memory and carries it forward.



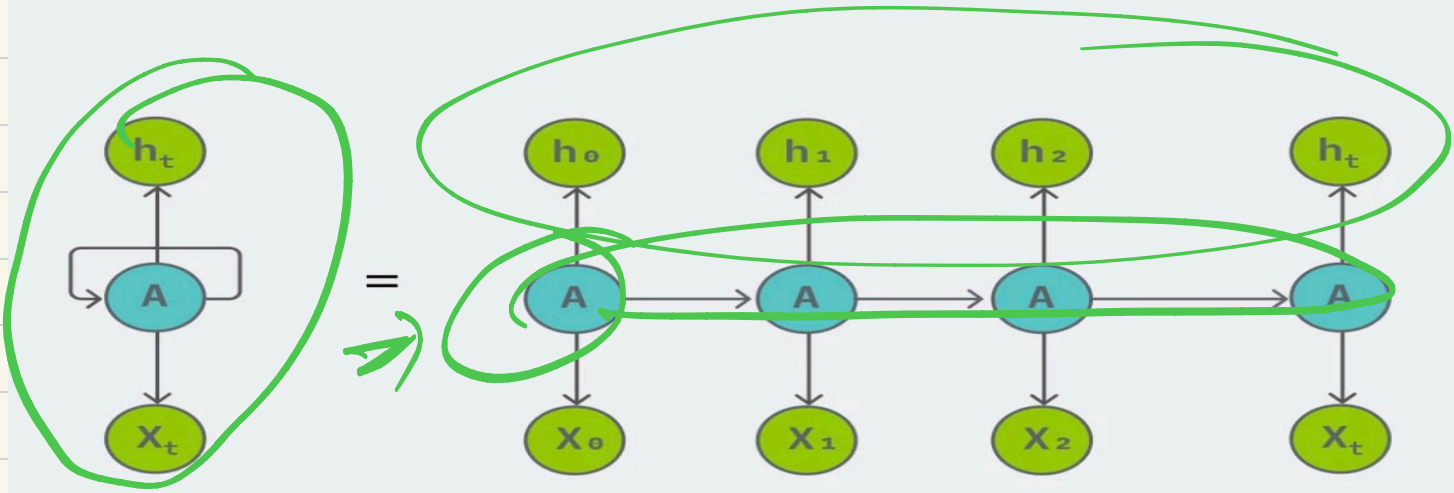
The Core Equation

Code

```
h_t  =  tanh( Wx · x_t  +  Wh · h_{t-1}  +  b )
ŷ_t  =  Wy · h_t
```

Breaking it down:

Symbol	Meaning
x_t	Current input at step t
h_{t-1}	Memory from previous step
h_t	New memory (updated hidden state)
Wx, Wh, Wy	Learned weight matrices
tanh	Activation — squashes values to (-1, +1)
ŷ_t	Output prediction at step t



🔁 How the Loop Works — Unrolled View

The best way to understand the loop is to **unroll** it across time. The same cell runs at each step, passing its hidden state forward like a baton in a relay race:

The cat

```
Code

h₀=0      h₁          h₂          h₃          h₄
  |         |           |           |           |
  ▼         ▼           ▼           ▼           ▼
[Cell 1]──→[Cell 2]──→[Cell 3]──→[Cell 4]──→[Cell 5]
  ▲         ▲           ▲           ▲           ▲
"The"     "cat"       "sat"       "on"       "the"
                                                  |
                                                  ▼
                                            Predict: "mat
```

What's happening step by step:

1. **t=1:** Feed "The" + h_0 (zeros) → produces h_1. Now h_1 "knows" about "The"
2. **t=2:** Feed "cat" + h_1 → produces h_2. Now h_2 knows "The cat"
3. **t=3:** Feed "sat" + h_2 → produces h_3. Memory growing
4. **t=4:** Feed "on" + h_3 → h_4 knows the whole phrase so far
5. **t=5:** Feed "the" + h_4 → final hidden state has full context → predicts "mat"

Types of RNN — Based on Input/Output Structure

The most important way to classify RNNs is by **how many inputs go in and how many outputs come out.**

1️⃣ One to One

This is actually just a **standard ANN** — not really RNN. Shown here for comparison.

```
Code

Input          Cell          Output

  x ──────────► [■] ──────────► ŷ

No sequence. No loop. One shot.
Example: Image classification (cat or dog?)
```

2️⃣ One to Many

One input → Multiple outputs over time

```
Code

                                                  (hidden state
 ──────►[Cell 1]──h₁──►[Cell 2]──h₂──►[Cell 3]──h₃──►[Cell 4]
                          │            │            │
                          ▼            ▼            ▼
                          ŷ₁           ŷ₂           ŷ₃

gle input feeds Cell 1 only.
aining cells only receive hidden state — no new x input.

mple: Music generation (give one seed note → generate full me
mple: Image captioning (give one image → generate full senter
```

↓

3 Many to One

Multiple inputs over time → Single output at the end

Code

```
x1          x2          x3          x4
 |           |           |           |
 ▼           ▼           ▼           ▼
[Cell 1]—h1→[Cell 2]—h2→[Cell 3]—h3→[Cell 4]
                                      |
                                      ▼
                                     ŷ
                                (only ONE output
                                 at the very end)

All inputs are consumed. Only final hidden state produces o

Example: Sentiment analysis ("This movie was absolutely ama
Example: Spam detection (read full email → spam or not spam
```

---

4 Many to Many (Synced)

Multiple inputs → Multiple outputs at every step

Code

```
x1          x2          x3          x4
 |           |           |           |
 ▼           ▼           ▼           ▼
[Cell 1]—h1→[Cell 2]—h2→[Cell 3]—h3→[Cell 4]
 |           |           |           |
 ▼           ▼           ▼           ▼
ŷ1          ŷ2          ŷ3          ŷ4

Input and output are in sync — one output per input step.

Example: POS Tagging
  Input:    "The    cat    sat    fast"
  Output:    DET    NOUN   VERB   ADV

Example: Video classificat ↓ (label each frame as it comes
```

5 Many to Many (Encoder-Decoder)

Multiple inputs first → then Multiple outputs

Also called **Seq2Seq**. This is the most powerful type.

```
Code

ENCODER PHASE                                  DECODER PHASE
(reading input)                                (generating output)

  x₁        x₂        x₃                       (no new x input)
  |         |         |
  ▼         ▼         ▼
[Cell]──►[Cell]──►[Cell]
                     |
                  context
                  vector C ──────►[Cell]──►[Cell]──►[Cell]
                                    |         |         |
                                    ▼         ▼         ▼
                                   ŷ₁        ŷ₂        ŷ₃

Encoder reads the full input and compresses into context ve
Decoder uses C to generate the output sequence step by step
```

6 Bidirectional RNN

BERT

Reads the sequence in BOTH directions simultaneously

```
Code

Forward RNN  →→→→→→→→→→→→→→→→→→→→→→→→→→→→→→→→

  x₁            x₂            x₃            x₄
  |             |             |             |
  ▼             ▼             ▼             ▼
[F-Cell]──►[F-Cell]──►[F-Cell]──►[F-Cell]
  |             |             |             |
  ▼             ▼             ▼             ▼
[B-Cell]◄──[B-Cell]◄──[B-Cell]◄──[B-Cell]
  |             |             |             |
  └─────┬───────┘             └─────┬───────┘
        ▼                           ▼
   concat(h_f, h_b)            concat(h_f, h_b)
        |                           |
        ▼                           ▼
        ŷ₁                          ŷ₄

←←←
```

Limitations of RNN — Why We Moved Beyond It

1️⃣ Vanishing Gradient Problem

The most famous and serious limitation.

During training, RNN learns by sending error signals **backward through time** (called BPTT — Backpropagation Through Time). The problem is that this gradient gets **multiplied repeatedly** at each step.

Code

```
FORWARD PASS (left to right — normal)

[Cell 1]──►[Cell 2]──►[Cell 3]──► ··· ──►[Cell 20]
                                          │
                                          Loss calculated
                                          here

BACKWARD PASS (right to left — gradient flow)

[Cell 1]◄──[Cell 2]◄──[Cell 3]◄── ··· ◄──[Cell 20]

Gradient at each step:
Step 20:  gradient = 1.000   ▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓
Step 15:  gradient = 0.100   ▓▓▓▓▓▓▓▓▓▓▓▓
Step 10:  gradient = 0.010   ▓▓▓▓▓▓
Step  5:  gradient = 0.001   ▓▓▓
Step  1:  gradient = 0.0001  ▒▒   ← almost zero
                                  ← Cell 1 learns NOTHING
```

0.002

Result: Early time steps stop learning. The network cannot capture long-range dependencies.

Code

```
"The cat, which was sitting on the mat, ___"
                                        ^
RNN has almost forgotten "c ↓ by the time it needs to predi
```

2 Exploding Gradient Problem

The opposite problem — gradients grow **too large** instead of shrinking.

```
Code

Gradient at each step (when weights > 1):

Step  1:  gradient = 1
Step  5:  gradient = 100
Step 10:  gradient = 10,000
Step 15:  gradient = 1,000,000  ← network weights go haywir
Step 20:  gradient = ∞          ← NaN errors, training cras
```

```
Code

Symptoms:
  → Loss suddenly jumps to NaN
  → Weights become astronomically large
  → Model completely fails to train

Fix used in practice:
  → Gradient Clipping (manually cap gradient at a threshold
  → But this is a band-aid, not a real fix
```

↓

3 Cannot Parallelize — Very Slow to Train

This is a **structural** limitation of RNN.

```
Code

ANN — Can Parallelize (fast ✅)

[Layer 1] ──→ [Layer 2] ──→ [Layer 3]
All neurons in a layer compute SIMULTANEOUSLY on GPU

──────────────────────────────────────────────

RNN — Cannot Parallelize (slow ❌)

[Cell 1]──→[Cell 2]──→[Cell 3]──→[Cell 4]──→[Cell 5]
    │           │
    └── must finish before next cell starts

Cell 2 cannot run until Cell 1 finishes.
Cell 3 cannot run until Cell 2 finishes.
...and so on.

Reason: each cell NEEDS h_{t-1} from the previous cell.
That dependency makes parallelization impossible.
```

Real world impact:

```
Code

Sentence: 100 words
  → RNN must run 100 sequential steps
  → On a 1000-word document? 1000 sequential steps
  → On a book? Extremely slow.

Transformer (GPT, BERT)?
  → Processes ALL words simultaneously
  → 100x faster training on GPUs
```

Fixed-Size Context Vector (Seq2Seq bottleneck)

In the Encoder-Decoder RNN, the entire input sequence is compressed into one single vector C.

ENCODER

"मैं कल दिल्ली जाऊंगा क्योंकि वहाँ मेरा काम है"

x_1 x_2 x_3 x_4 x_5 x_6 x_7 x_8

[Cell] → [Cell] → [Cell] → [Cell] → [Cell] → [Cell] → [Cell] → [Cell]

Context Vector C
[one fixed vector]

DECODER uses only C to generate full English translation

Problem:
C must store ALL information from ALL 8 words.
For long sentences → C gets overloaded → information is lost.
Translation quality DROPS as sentence length increases.